IMPLEMENTING AND EVALUATING POST-QUANTUM KEY ESTABLISHMENT IN TELEMETRY SYSTEMS

Yuxing Yang*
*School of Information Sciences, University of Illinois Urbana-Champaign, Champaign, IL
**Information Trust Institute, University of Illinois Urbana-Champaign, Champaign, IL
E-mails: yuxingy2@illinois.edu, dsaeb2@illinois.edu

Faculty Advisor:
Dominic Saebeler**

ABSTRACT

Telemetry systems require secure key establishment to protect transmitted data. The development of quantum computers creates a need for post-quantum cryptography (PQC) to replace current asymmetric algorithms. This paper presents the implementation and evaluation of post-quantum key establishment in a telemetry communication setting. A software-based test bed is developed to simulate communication between a Test Article (TA) and a Ground Station (GS). The system uses both classical Elliptic Curve Diffie-Hellman (ECDH) and post-quantum ML-KEM. The test bed enables repeated experiments and measures system-level performance, including latency, CPU time, memory usage, and communication overhead. The proposed system provides a platform for analyzing the impact of post-quantum cryptography on telemetry systems.

INTRODUCTION

Telemetry systems need secure key establishment before mission data can be protected. This step sits at the start of every secure telemetry session. A Ground Station and a Test Article must agree on a shared secret before any data gets encrypted. So the cost of this step is not a small setup detail. It is part of the operational cost of the whole system.

Quantum computers change what counts as secure here. Classical key exchange methods like Elliptic Curve Diffie Hellman work well today. But they cannot protect data over the long run against a future quantum computer [1]. NIST responded by standardizing ML-KEM at three security levels. Hybrid schemes also exist. They combine a classical exchange with a post-quantum step, so a system does not rely on only one new method during the switch. These options give operators more choices. But each choice comes with its own payload size, compute cost, and latency profile.

An organization planning this switch needs real numbers, not just theory. It needs to know what each option actually costs once it runs inside a real protocol stack, under real network conditions, and under real server load. This sounds simple. But measuring it correctly is harder than it looks.

Two different algorithms often run through two different code libraries. If those libraries carry different overhead on their own, the comparison stops being fair. A result can look like an algorithm problem when it is really an implementation problem. This paper treats that risk as a central part of the measurement task, not as a side note.

This paper presents a testbed that measures classical ECDH and post-quantum ML-KEM inside a Ground Station to Test Article link. All schemes run through the same TLS 1.3 stack. This keeps the comparison focused on the algorithm, not on the software wrapped around it. The testbed records latency, CPU time, memory usage, and communication overhead across a wide range of network and load conditions. This gives a full picture of what post-quantum migration actually costs, not just a single number under one condition.

This paper makes three contributions. First, it identifies a common source of measurement bias in cross-algorithm benchmarking and shows how to remove it. Second, it builds a testbed that measures handshake cost under realistic round trip time, bandwidth, and concurrent load conditions, not only on an idle local link. Third, it uses this testbed to produce a full performance profile for post-quantum and hybrid key establishment in a telemetry setting. This profile can support real migration planning, not only a single benchmark result.

BACKGROUND

A. Key Establishment in Telemetry Systems

Telemetry systems protect mission data with symmetric encryption. This starts only after a session key gets set up between the Ground Station and the Test Article. So the key exchange step sits at the front of the whole secure pipeline. Classical public key methods handle this step today. RSA and Elliptic Curve Diffie Hellman are the most common. These methods run fast and work well now. But they cannot hold up over the long term. A powerful enough quantum computer could break them using Shor's algorithm [1]. Symmetric encryption faces less risk from this shift. Grover's algorithm only gives a quadratic speedup against it. Operators usually respond by using longer symmetric keys [2].

This is why post-quantum migration work focuses so heavily on the handshake layer, not the data layer. Telemetry adds its own limits on top of this. A handshake must fit inside a short contact window. It must run within tight link bandwidth. It must respect ground station processing limits too. Recent ITC work has studied post-quantum key encapsulation for aeronautical mobile telemetry. This work looked at code-based methods for sending symmetric keys between telemetry parties [3]. IRIG 106 also sets standards for how telemetry links and networks must operate [4]. Any new key exchange method has to work inside these existing rules. This shows why telemetry needs its own study of post-quantum key exchange. Results from general internet protocols do not always carry over cleanly.

B. NIST Post-Quantum Standardization and Hybrid Migration

NIST built ML-KEM from the CRYSTALS-Kyber submission. It standardized ML-KEM at three security levels [5, 6]. These levels give operators a choice between different security margins and different performance costs. Higher levels give stronger protection. They also carry larger keys and larger ciphertexts.

Hybrid key exchange offers another path through this transition. A hybrid scheme runs a classical exchange next to a post-quantum encapsulation on the same handshake [7]. This way, the session does not depend on only one new method during migration. Formal work on hybrid key encapsulation gives this design a solid security foundation [8]. The benefit is migration safety. The cost is that each handshake now carries more data and does more cryptographic work than a single classical or post-quantum exchange alone. This cost is easy to state but hard to measure well. The next section explains why.

C. Measurement Challenges in Cross-Algorithm Benchmarking

Measuring the true cost of a key exchange scheme sounds straightforward. Run the handshake, record the time, compare the numbers. But this only works if every scheme runs through a fair and matched setup. Prior benchmarking work shows that post-quantum overhead in TLS depends heavily on more than just the algorithm. Paquin, Stebila, and Tamvada benchmarked several post-quantum KEMs inside TLS. They found that the measured cost shifted with the protocol setting and the network environment, not only the choice of algorithm [9]. Sikeridis, Kampanakis, and Devetsikiotis found a similar pattern for post-quantum TLS authentication. Overhead swung by a wide margin across different link conditions [10].

A separate risk sits underneath both of these findings. Different algorithms often run through different software libraries during a benchmark. Classical ECDH might run through one cryptography library. A post-quantum scheme might run through a separate library built on a different provider. If these libraries carry different binding overhead on their own, this overhead gets mixed into the result. A scheme can then look slower or faster than it really is, for reasons that have nothing to do with the algorithm itself. This risk grows when a benchmark reports results that will guide a real migration decision. A biased number does not just look wrong on a chart. It can point an operator toward the wrong plan. The Open Quantum Safe project addresses part of this problem by giving post-quantum algorithms a common integration path into standard tools like OpenSSL [11]. Section IV shows how this paper uses that same idea to remove implementation bias from the testbed design.

TESTBED DESIGN

A. Ground Station to Test Article Model

This paper considers a Ground Station, denoted GS, and a Test Article, denoted TA. The two sides run a key exchange handshake before any mission data moves between them. The handshake

creates a shared secret. The system uses this secret to protect the telemetry stream that follows. Mission data cannot move under protection until this step finishes. So the time this step takes is not a background cost. It sits directly on the critical path of the telemetry session.

The testbed places the GS and TA on the same machine at first, then adds controlled network conditions between them. This setup lets the same handshake run many times under many different link states. It also lets every scheme run under the exact same conditions, so differences in the result come from the scheme, not from a shifting test environment.

B. Measured Configurations

The testbed measures seven key exchange configurations. These are classical ECDH, ML-KEM at three security levels, and three hybrid combinations that pair ECDH with each ML-KEM level. Table 1 lists each configuration along with its NIST security level.

Table 1: Measured key exchange configurations.

Configuration	Type	Security Level
ECDH	Classical	N/A
ML-KEM-512	Post-quantum	Level 1
ML-KEM-768	Post-quantum	Level 3
ML-KEM-1024	Post-quantum	Level 5
Hybrid-512	Hybrid	Level 1
Hybrid-768	Hybrid	Level 3
Hybrid-1024	Hybrid	Level 5

Classical ECDH gives a baseline for the current, pre-quantum deployment. The other six configurations cover the full range of post-quantum and hybrid options that a telemetry operator could adopt.

C. Avoiding Implementation Bias in Cross-Algorithm Benchmarking

An early version of this testbed measured classical ECDH through one Python cryptography library and measured ML-KEM through a separate library built on liboqs. Both libraries wrapped the same underlying handshake logic in Python. This setup looked reasonable at first. Each library was a standard, well-supported choice for its algorithm.

The early results did not match expectations. Classical ECDH showed a mean handshake latency roughly three times higher than ML-KEM, despite carrying a public key and ciphertext far smaller than any ML-KEM configuration. A smaller key with more computational simplicity should not produce a slower result on the same hardware. This gap pointed to a problem with the measurement, not with the algorithm.

Closer inspection traced the gap to the two Python libraries themselves. Each library added its own layer of Python-level overhead on top of the underlying cryptographic call. These two overhead

layers were not the same size. So part of the measured latency difference came from the wrapping code, not from ECDH or ML-KEM. Comparing the two schemes this way meant comparing two different software stacks as much as comparing two different algorithms. A benchmark built this way could point a real operator toward the wrong conclusion about which scheme actually costs less.

The testbed was redesigned to remove this source of bias. Every configuration now runs as a real TLS 1.3 server, built with OpenSSL, a widely deployed and independently studied TLS implementation [12], and the Open Quantum Safe provider [11]. Each server is locked to one key exchange group using the OpenSSL `-groups` setting, so a single connection always negotiates one exact configuration. The client side connects using Python's standard `ssl` module. Every configuration, classical or post-quantum, goes through this same client code path. No configuration receives special handling. This design keeps the comparison focused on the algorithm itself, since every scheme now runs through one shared protocol stack instead of separate library wrappers with unmatched overhead. This distinction matters beyond this testbed. Any benchmark that measures classical and post-quantum schemes through different software paths risks producing the same kind of misleading result.

D. Network and Concurrency Emulation

The testbed also needs to measure handshake cost under realistic network and load conditions, not only on an idle local link. A transparent proxy sits between the GS and TA connection. It adds a one-way propagation delay equal to half the target round trip time on each direction. It also applies a token-bucket rate limit to cap bandwidth. This setup reproduces the effect of `tc netem` without requiring elevated system privileges.

Concurrent load is produced using a thread barrier. All client threads wait at the barrier, then start their handshake at the same instant. This creates a true burst of simultaneous connection requests at the server, rather than a sequence of requests spaced out over time. Every configuration is measured under the same emulated network and load conditions, using the same proxy and the same barrier mechanism.

EXPERIMENTAL METHODOLOGY

A. Testbed Configuration

Table 2 lists the hardware and software configuration used for every measurement in this paper. Every configuration listed in Table 1 runs on this same setup, so no scheme receives an advantage from a different environment.

B. Measurement Procedure and Statistical Method

Each experiment repeats every configuration many times under the same condition before moving to the next condition. The single-handshake baseline experiment runs 100 trials per configura-

Table 2: Testbed hardware and software configuration.

Component	Configuration
CPU	Apple M-series, 8-core
Operating system	macOS 15
TLS library	OpenSSL 3.6
PQC provider	Open Quantum Safe (oqs-provider)
Protocol version	TLS 1.3
Client library	Python `ssl` module
Network emulation	Proxy-based delay and rate limiting
Concurrency model	Thread barrier synchronization

tion. The network sensitivity experiment runs 30 trials per configuration at each combination of round trip time and bandwidth. The concurrency experiment runs each configuration once at each concurrency level, using the full set of simultaneous client threads described in Section IV-D.

Measurement order is not fixed across configurations. Trials for different schemes are interleaved rather than run in a fixed block for each scheme. This reduces the risk that a result reflects a change in background system load, thermal state, or other time-dependent factors rather than a real property of the scheme itself.

Latency is reported as the mean across all trials for a given condition, along with a 95% confidence interval computed using the t-distribution. This gives a sense of how much a reported mean could shift if the experiment were repeated. Tail latency is reported as the 99th percentile, denoted P99, across the same set of trials. P99 matters for telemetry because a small share of slow handshakes can still delay the point where mission data starts moving under protection, even if most handshakes complete quickly.

C. Metrics

Four metrics are recorded for every handshake. Wall-clock latency measures the full time from the start of the connection to the point where the TLS session is ready for application data. CPU time measures the processing time spent by the client process during the handshake, separate from time spent waiting on the network. Memory usage measures the change in resident memory before and after the handshake. Byte count measures the size of the key exchange payload sent in each direction, taken from the TLS key share record.

These four metrics together give a full account of what each configuration costs. Latency alone does not show whether a slower result comes from extra computation, extra network transmission, or both. Reporting all four metrics side by side lets a reader trace a performance difference back to its actual source.

RESULTS

A. *Baseline Handshake Cost*

Table 3 reports mean latency, CPU time, and handshake payload size for each configuration under no added network delay and a single client connection. This condition isolates the raw cost of each scheme, separate from any network or load effect.

Table 3: Baseline handshake cost, single client, no added delay ($n = 100$).

Configuration	Mean (ms)	95% CI	CPU (ms)	Bytes
ECDH	28.25	0.72	21.69	130
ML-KEM-512	27.74	0.42	21.73	1568
ML-KEM-768	29.21	0.96	22.33	2272
ML-KEM-1024	31.64	2.43	23.14	3136
Hybrid-512	35.41	0.41	26.84	1698
Hybrid-768	30.00	1.00	23.06	2402
Hybrid-1024	30.92	1.24	23.40	3233

Every configuration now falls within a narrow band between 27.7 ms and 35.4 ms. This is a direct consequence of the testbed redesign described in Section IV-C. Under the earlier implementation, classical ECDH showed a mean latency roughly three times higher than the post-quantum schemes, despite carrying the smallest handshake payload of any configuration. Under the unified TLS 1.3 stack, ECDH and ML-KEM-512 now differ by less than 2%, and no configuration exceeds ECDH by more than 25%. Handshake payload size still grows sharply with security level, from 130 bytes for ECDH to 3233 bytes for Hybrid-1024, but this growth in payload does not translate into a matching growth in latency. This confirms that the earlier three-fold gap was an artifact of the measurement setup, not a property of the algorithms themselves.

B. *Sensitivity to Round-Trip Time and Bandwidth*

Table 4 reports mean latency for six configurations under two contrasting conditions: a low round-trip time with a tightly constrained 1 Mbps link, and a high round-trip time with an unconstrained link.

Table 4: Mean latency (ms) under contrasting network conditions.

Configuration	Δ=0ms, W=1Mbps	Δ=200ms, W=∞
ECDH	37.6	599.4
ML-KEM-512	39.1	583.1
ML-KEM-768	40.6	581.5
ML-KEM-1024	43.8	602.1
Hybrid-512	41.4	593.0
Hybrid-1024	43.2	591.8

When round-trip time dominates, the six configurations converge to within 21 ms of one another, all near 590 ms. Payload size differences that reach 3100 bytes between ECDH and Hybrid-1024

become invisible once propagation delay sets the floor for total latency. Under a constrained 1 Mbps link with no added propagation delay, the same configurations spread further apart, from 37.6 ms for ECDH to 43.8 ms for ML-KEM-1024, a 16% range compared to a 4% range under high RTT. Bandwidth-limited conditions are therefore where payload size actually shapes the result. A telemetry link operating on a long-delay path, such as a geostationary relay, will see little practical difference between configurations. A link constrained by bandwidth, such as an early low-Earth-orbit pass, is where the choice of security level carries a measurable cost.

C. Sensitivity to Concurrent Load

Table 5 reports mean and P99 latency for ML-KEM-768 and Hybrid-512 across three concurrency levels. These two configurations are highlighted because they represent the lowest-latency post-quantum option and the most commonly recommended hybrid transition option, respectively.

Table 5: Mean and P99 latency under concurrent client load.

n	ML-KEM-768 Mean	Hybrid-512 Mean	ML-KEM-768 P99	Hybrid-512 P99
50	132.3 ms	156.9 ms	176.4 ms	224.1 ms
100	215.2 ms	324.7 ms	321.5 ms	469.4 ms
200	558.2 ms	764.0 ms	1401.5 ms	1602.8 ms

The gap between the two configurations grows with concurrency. Hybrid-512 mean latency exceeds ML-KEM-768 by 18.6% at $n = 50$, by 50.9% at $n = 100$, and by 36.9% at $n = 200$. The P99 gap follows the same direction and grows faster than the mean gap at every level. This shows that the added cost of running both a classical and a post-quantum step on every handshake falls most heavily on the tail of the latency distribution rather than the typical case.

Table 6 extends this comparison across all six post-quantum and hybrid configurations, using ML-KEM-768 as the reference point since it produces the lowest latency under concurrent load.

Table 6: ML-KEM-768 advantage over each configuration, averaged over $n = 50, 100, 200$.

Configuration	Mean Advantage	P99 Advantage
ECDH	19.6%	9.6%
ML-KEM-512	8.7%	0.7%
ML-KEM-1024	8.3%	4.2%
Hybrid-512	27.3%	17.3%
Hybrid-768	14.8%	5.3%
Hybrid-1024	17.2%	7.6%

The largest advantage appears against the hybrid configurations, not against classical ECDH. This is the opposite pattern from what a purely compute-focused view of migration would predict. Under concurrent load, the cost that matters most is not classical versus post-quantum. It is single-exchange versus dual-exchange, since a hybrid handshake performs the work of two schemes on every connection, and that added work compounds once the server must service many handshakes at once.

D. Comparison Against Production TLS 1.3

Table 7 compares each configuration against a production TLS 1.3 server running with no forced key exchange group, representing a standard, unmodified deployment.

Table 7: Mean latency versus a production TLS 1.3 baseline ($n = 100$ each).

Configuration	Mean Latency (ms)
Production TLS 1.3 (default)	29.42
ECDH	28.25
ML-KEM-768	29.21
Hybrid-512	35.41

Every measured configuration falls within the range spanned by a standard production TLS 1.3 deployment, which itself varies by more than 4 ms across its own 100 trials. ML-KEM-768 lands within 0.2 ms of the production baseline. Hybrid-512, the slowest configuration measured, still adds only 6 ms over the production baseline, a difference smaller than the natural run-to-run variation already present in a standard, unmodified TLS 1.3 server. Post-quantum migration cost at the handshake layer is not detectable above the variation a telemetry operator would already see from one production TLS session to the next.

DISCUSSION

A. What the Measurements Imply for Migration Planning

The results in Section V point to a conclusion that runs against a common assumption. Migration cost is often framed as a computation problem: post-quantum algorithms are assumed to be slower, so migration is assumed to carry a performance penalty. Table 3 shows that this framing does not hold once the comparison runs through a matched software stack. ECDH and ML-KEM-512 differ by less than 2% at baseline. The real cost sits somewhere else.

Section V-B shows where. Bandwidth is the condition that actually separates the configurations, not round-trip time. A link with generous bandwidth will barely notice a switch to post-quantum key exchange, at any security level. A link constrained to 1 Mbps will notice, and the gap will grow with the security level chosen. This matters for planning because bandwidth constraints are not evenly distributed across a mission. A ground station with a dedicated high-bandwidth uplink faces a different decision than an early-pass link running on a fraction of that capacity.

Section V-C shows a second cost that has nothing to do with bandwidth. Concurrent load exposes a cost that single-connection tests cannot see. Hybrid-512 costs 27% more than ML-KEM-768 on average once 50 or more handshakes arrive together, and that gap holds at P99 as well as at the mean. A ground station evaluating migration on a single test handshake would miss this cost entirely. It only appears once the station is measured under the load it will actually carry.

B. Deployment Guidance by Operating Condition

Table 8 summarizes deployment guidance drawn directly from the measurements in Section V, organized by the operating condition that dominates each case.

Table 8: Deployment guidance by dominant operating condition.

Condition	**Cost driver**	**Guidance**
High RTT, ample bandwidth	Propagation delay dominates	Any configuration is acceptable
Low RTT, constrained bandwidth	Payload size dominates	Prefer lower security level or PQ-only
High concurrency	Dual-exchange compute dominates	Prefer PQ-only over hybrid
Low concurrency, transition period	Neither dominates	Hybrid remains a reasonable choice

No single configuration is correct across every row of Table 8. This is the main practical finding of this paper. A migration plan built around one fixed configuration will be well matched to some operating conditions and poorly matched to others. A plan informed by measurements across the full range of conditions a system will actually encounter can instead match the configuration to the condition.

C. Limitations

Three limitations bound these results. First, all measurements come from a software testbed running on general-purpose hardware, not flight hardware. Embedded processors with tighter compute and memory budgets may show a different balance between classical and post-quantum cost than the hardware used here. Second, network conditions are controlled through a fixed set of round-trip time and bandwidth values, not continuous real-world variation. A live telemetry link may include packet loss, jitter, and gradual bandwidth drift that this testbed does not reproduce. Third, the concurrency results in Section V-C represent a single trial per configuration at each load level; run-to-run variance at high concurrency was not independently quantified and may affect the precision of the reported percentages. The concurrency tests also apply a synchronized burst of simultaneous connections, representing a worst-case arrival pattern; real handshake arrivals may be less tightly clustered, which would further narrow the gap reported in Section V-C.

CONCLUSION

This paper presented a testbed for measuring post-quantum key establishment cost in a telemetry setting. The testbed measures classical ECDH, ML-KEM at three security levels, and three hybrid configurations, all running through the same TLS 1.3 stack. This design choice removed a source of measurement bias found during an earlier version of the testbed, where classical and post-

quantum schemes ran through separate software libraries with unmatched overhead. That earlier setup produced a result that looked like an algorithm problem but was really an implementation problem. The corrected testbed shows that classical and post-quantum handshakes differ by less than 2% under matched conditions.

The measurements show that migration cost is not primarily a computation problem. It is a bandwidth problem and a concurrency problem. Payload size differences become visible only when bandwidth is constrained and round-trip time is low. Hybrid schemes carry a cost that single-connection tests cannot see, one that only appears once many handshakes arrive together and grows to 27% at the mean and 17% at the tail under sustained concurrent load. Neither of these costs would show up in a benchmark that measures one handshake on an idle link.

These findings support a general point about how post-quantum migration should be evaluated. A single latency number, measured under one condition, does not describe what a real deployment will experience. The conditions that matter, bandwidth, round-trip time, and concurrent load, vary across a mission and across a telemetry network. A measurement approach that spans this range gives an operator a basis for planning migration around the conditions a system will actually face, rather than around a single benchmark result.

Future work should extend this testbed to flight-representative hardware and to network traces drawn from real telemetry passes, rather than a fixed grid of controlled conditions. This would show whether the cost patterns identified here hold under the packet loss, jitter, and gradual bandwidth drift that a live link introduces, and whether the concurrency costs measured here persist under less tightly synchronized handshake arrival patterns.

ACKNOWLEDGMENTS

The author thanks Dominic Saebeler for his guidance and mentorship as faculty advisor throughout this project. Portions of the testbed code, including the network delay/bandwidth emulation, concurrent load testing scripts, and statistical analysis routines, were developed with the assistance of Claude Sonnet 5 (Anthropic). The author designed all experiments, interpreted the results, and is solely responsible for the conclusions presented in this paper.

REFERENCES

[1] P. W. Shor, “Algorithms for quantum computation: Discrete logarithms and factoring,” in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, IEEE, 1994.

[2] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pp. 212–219, 1996.

[3] B. Arnesen, T. Wai, T. Randall, Y. Yang, D. Mitchell, and W. K. Harrison, "Applying post-quantum cryptography based on quasi-cyclic codes to aeronautical mobile telemetry," in *Proceedings of the International Telemetering Conference*, vol. 60, 2025.

[4] "Telemetry standards," IRIG 106-25, Telemetry Group, Range Commanders Council, 2025.

[5] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard (fips 203)," Federal Information Processing Standard 203, NIST, 2024.

[6] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "Crystals-kyber: A cca-secure module-lattice-based kem," *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pp. 353–367, 2018.

[7] J. M. Schanck, W. Whyte, and Z. Zhang, "Hybrid key exchange in tls 1.3." Internet-Draft, IETF, 2020. draft-schanck-tls-additional-keyshare-00.

[8] N. Bindel, J. Brendel, M. Fischlin, B. Gonçalves, and D. Stebila, "Hybrid key encapsulation mechanisms and authenticated key exchange," in *Post-Quantum Cryptography–PQCrypto 2019*, pp. 206–226, Springer, 2019.

[9] C. Paquin, D. Stebila, and G. Tamvada, "Benchmarking post-quantum cryptography in tls," in *Post-Quantum Cryptography: 11th International Conference, PQCrypto 2020*, pp. 72–91, Springer, 2020.

[10] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Post-quantum authentication in tls 1.3: A performance study," in *Network and Distributed System Security Symposium*, 2020.

[11] D. Stebila and M. Mosca, "Post-quantum key exchange for the internet and the open quantum safe project," in *Selected Areas in Cryptography–SAC 2016*, pp. 14–37, Springer, 2017.

[12] B. Beurdouche, K. Bhargavan, A. Delignat-Lavaud, C. Fournet, M. Kohlweiss, A. Pironti, P.-Y. Strub, and J. K. Zinzindohoue, "A messy state of the union: Taming the composite state machines of tls," in *2015 IEEE Symposium on Security and Privacy*, pp. 535–552, IEEE, 2015.